Guía 2 de 3 — Guardar en una base de datos: SQLite y cargas repetibles

DPL1046 · Clase 6 · Para repasar en casa, a tu ritmo.

Un CSV sirve, pero una **base de datos** es mucho más cómoda para guardar y consultar resultados. En esta guía repasas **SQLite**, un poco de **SQL** y la idea de **no duplicar datos** cuando corres un script varias veces.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado.

---

1. ¿Por qué una base de datos y no solo un CSV?

Un CSV es como un **fajo de hojas sueltas**: sirve, pero para buscar algo hay que leerlo todo. Una base de datos es un **archivador con índice**: preguntas lo que necesitas y responde rápido. **SQLite** es la más simple: **toda la base vive en un solo archivo** (por ejemplo `data/curso.db`) y viene incluida con Python. Lo que aprendas de SQL sirve para cualquier otra base.

---

2. De pandas a SQLite

```
import sqlite3
import pandas as pd

RUTA = "data/raw/"
pagos = pd.read_csv(RUTA + "olist_order_payments_dataset.csv")

conexion = sqlite3.connect("data/curso.db")
pagos.to_sql("pagos", conexion, if_exists="replace", index=False)
print("Filas guardadas:", len(pagos))                # 103886
```

- `sqlite3.connect("data/curso.db")` abre la base y, si no existe, la **crea**. `conexion` es el "cable" entre Python y el archivo; como abrir la puerta del archivador.
- `to_sql("pagos", conexion, if_exists="replace", index=False)` guarda la tabla de pandas **dentro de la base**, con el nombre `pagos`. `index=False` evita guardar los números 0, 1, 2…

Consultar con SQL

```
consulta = """
    SELECT payment_type, COUNT(*) AS cantidad, ROUND(SUM(payment_value)) AS total
    FROM pagos
    GROUP BY payment_type
    ORDER BY total DESC
"""
resultado = pd.read_sql_query(consulta, conexion)
print(resultado)

conexion.close()
```

Tres comillas dobles permiten escribir texto en **varias líneas**. El SQL se lee casi como inglés:

- `SELECT payment_type, COUNT(*) AS cantidad, ROUND(SUM(payment_value)) AS total` → "elige el medio de pago, **cuenta** las filas (la llamo `cantidad`) y **suma** el valor (la llamo `total`)".
- `FROM pagos` → "desde la tabla `pagos`".
- `GROUP BY payment_type` → "agrupando por medio de pago".
- `ORDER BY total DESC` → "ordenado por total, de mayor a menor".
- `pd.read_sql_query(consulta, conexion)` manda la consulta y **recibe una tabla de pandas**.
- `conexion.close()` cierra la conexión, como cerrar el archivador.

---

3. SQL mínimo: el diccionario con pandas

Quiero...	SQL	pandas
Elegir columnas	`SELECT a, b`	`tabla[["a", "b"]]`
Filtrar filas	`WHERE precio > 500`	`tabla[tabla["precio"] > 500]`
Agrupar	`GROUP BY estado`	`tabla.groupby("estado")`
Contar	`COUNT(*)`	`.size()`
Sumar	`SUM(valor)`	`.sum()`
Ordenar	`ORDER BY total DESC`	`.sort_values("total", ascending=False)`
Primeras filas	`LIMIT 5`	`.head(5)`
Unir tablas	`JOIN ... ON ...`	`.merge(..., on=...)`

Son las **mismas operaciones** que ya haces con pandas: aprender SQL es aprender otra forma de escribirlas.

---

4. Correr dos veces, mismo resultado (idempotencia)

Un **interruptor de luz**: si lo pulsas dos veces en "encendido", la luz sigue encendida. Un **timbre**, en cambio, suena cada vez. Un script **idempotente** deja el mismo resultado sin importar cuántas veces lo corras. Mientras pruebas lo vas a correr varias veces: si cada vez *agrega* los datos, tus totales se triplican.

```
def contar_filas():
    """Cuenta las filas que hay hoy en la tabla 'pagos'."""
    return pd.read_sql_query("SELECT COUNT(*) AS n FROM pagos", conexion)["n"][0]

pagos.to_sql("pagos", conexion, if_exists="replace", index=False)
print("Primera carga:", contar_filas())             # 103886

pagos.to_sql("pagos", conexion, if_exists="append", index=False)
print("Con append (duplica):", contar_filas())      # 207772

pagos.to_sql("pagos", conexion, if_exists="replace", index=False)
print("Con replace (no duplica):", contar_filas())  # 103886
```

- `def contar_filas():` define una **función** (una receta con nombre; la veremos en la Clase 7).
- `return` devuelve el resultado a quien la llamó.
- `if_exists="replace"` → parte de cero cada vez: **no duplica**.
- `if_exists="append"` → agrega otra copia de todo: **duplica**.
- `if_exists="fail"` (el valor por defecto) → da error si la tabla ya existe.

`if_exists`	Qué hace	¿Duplica al correr 2 veces?
`replace`	Borra la tabla y la crea de nuevo	No
`append`	Agrega las filas al final	**Sí**
`fail`	Avisa con un error	No (pero falla)

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
`OperationalError: no such table`	La tabla aún no se guardó o escribiste mal el nombre	Corre antes el script que la crea; revisa el nombre
`database is locked`	Otra conexión quedó abierta	Cierra con `conexion.close()` y reinicia el script
`ValueError: Table 'pagos' already exists`	Usaste `if_exists="fail"`	Usa `replace` (o `append` si realmente quieres sumar)
Los totales se duplican	Usaste `append` varias veces	Cambia a `replace` o filtra lo que ya existe
`no such column` en el SQL	Escribiste mal una columna	Compara con `pagos.columns`

Practica tú

1. Guarda la tabla `orders` en una tabla SQLite llamada `pedidos`.
2. Con SQL, cuenta los pedidos por `order_status`.
3. ¿Qué `if_exists` usarías en un script que se corre todos los días y debe dejar siempre el mismo resultado?

▼ **Ver respuestas**

1. `orders.to_sql("pedidos", conexion, if_exists="replace", index=False)`
2. `pd.read_sql_query("SELECT order_status, COUNT(*) AS n FROM pedidos GROUP BY order_status", conexion)`
3. `replace`: no duplica.

Para llevarte

- SQLite = una base de datos en **un archivo**, incluida con Python.
- `to_sql` guarda; `read_sql_query` consulta con SQL y devuelve una tabla de pandas.
- SQL y pandas hacen lo mismo con distinta escritura.
- **Idempotente** = correr dos veces deja el mismo resultado. `replace` en vez de `append`.